

ITA06 - Machine Learning

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Aim

To implement and demonstrate the FIND-S algorithm in Python for finding the most specific hypothesis from a given set of training data samples.

Procedure

1. Collect the training dataset containing input attributes and the target class.
2. Initialize the hypothesis with the most specific values, represented as 0 or $\emptyset$.
3. Read each training sample one by one.
4. Check the target/class value of each sample.
5. If the sample is positive (Yes), compare its attributes with the current hypothesis.
6. If an attribute in the hypothesis is 0, replace it with the corresponding attribute value from the positive sample.
7. If the attribute value is different from the current hypothesis, replace it with ?.
8. If the sample is negative (No), ignore it.
9. Repeat the process for all training samples.
10. Display the final most specific hypothesis obtained from the positive training examples.
11. Implement the above steps using Python in Google Colab and verify the output.

Code

```
import pandas as pd

# Play Tennis dataset from the Kaggle notebook
data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
```

```

['Rain', 'Mild', 'High', 'Weak', 'Yes'],
['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
['Rain', 'Cool', 'Normal', 'Strong', 'No'],
['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
['Sunny', 'Mild', 'High', 'Weak', 'No'],
['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
['Rain', 'Mild', 'High', 'Strong', 'No']
]

columns = ['Outlook', 'Temperature', 'Humidity', 'Wind', 'PlayTennis']

df = pd.DataFrame(data, columns=columns)

print("PLAY TENNIS DATASET")
print("=====")
print(df.to_string(index=False))

# FIND-S Algorithm

attributes = columns[:-1]
target = columns[-1]

h = ['0'] * len(attributes)

print("\nInitial Hypothesis:")
print(h)

print("\nHypothesis after each positive example:")

```

```

print("-----")

for index, row in df.iterrows():

    if row[target] == 'Yes':

        for i in range(len(attributes)):

            if h[i] == '0':
                h[i] = row[attributes[i]]

            elif h[i] != row[attributes[i]]:
                h[i] = '?'

        print("Example", index + 1, ":", h)

    print("\nFinal Most Specific Hypothesis:")
    print(h)

```

Output

Initial Hypothesis:

['0', '0', '0', '0']

Hypothesis after each positive example:

Example 3 : ['Overcast', 'Hot', 'High', 'Weak']

Example 4 : ['?', '?', 'High', 'Weak']

Example 5 : ['?', '?', '?', 'Weak']

Example 7 : ['?', '?', '?', '?']

Example 9 : ['?', '?', '?', '?']

Example 10 : ['?', '?', '?', '?']

Example 11 : ['?', '?', '?', '?']

Example 12 : ['?', '?', '?', '?']

Example 13 : ['?', '?', '?', '?']

Final Most Specific Hypothesis:

['?', '?', '?', '?']

Result

Thus, the FIND-S algorithm was implemented successfully, and the most specific hypothesis obtained from the given training data was ['?', '?', '?', '?'].

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training Example

AIM

To implement and demonstrate the Candidate-Elimination algorithm in Python using training data examples stored in a CSV file, and to obtain the set of all hypotheses that are consistent with the given training examples.

Procedure

1. Load the training dataset from the given .CSV file using Python.
2. Separate the input attributes and target/class attribute from the dataset.
3. Initialize the Specific boundary (S) with the most specific hypothesis.
4. Initialize the General boundary (G) with the most general hypothesis.
5. Read the training examples one by one.
6. For a positive example, generalize the Specific boundary so that it covers the example and remove inconsistent hypotheses from the General boundary.
7. For a negative example, specialize the General boundary so that it excludes the example and remains consistent with the Specific boundary.
8. Remove hypotheses that are more general or more specific than required.
9. Continue the process until all training examples have been processed.
10. Display the final Specific boundary (S) and General boundary (G).
11. The hypotheses between these two boundaries represent the set of all hypotheses consistent with the training examples.

Code

```
import pandas as pd

from itertools import product

import glob

# Find the uploaded CSV file
files = glob.glob('/content/*.csv')

if len(files) == 0:
```

```

print("No CSV file found. Please upload your dataset.")
else:
    file_path = files[0]
    print("Using file:", file_path)

    data = pd.read_csv(file_path)

    print("\nTraining Dataset:")
    print(data.to_string(index=False))

    # Attributes and target
    attributes = list(data.columns[:-1])
    target = data.columns[-1]

    concepts = data[attributes].astype(str).values
    labels = data[target].astype(str).str.strip().str.lower().values

    n = len(attributes)

    # Values available for each attribute
    domains = []
    for i in range(n):
        domains.append(sorted(data[attributes[i]].astype(str).unique()))

    # Check whether hypothesis h covers example x
    def covers(h, x):
        for i in range(n):
            if h[i] != '?' and h[i] != 'Ø' and h[i] != x[i]:
                return False
        return True

```

```
# h1 is more general than h2
```

```
def more_general(h1, h2):
```

```
    for a, b in zip(h1, h2):
```

```
        if a != '?' and a != b:
```

```
            return False
```

```
    return True
```

```
# Minimal generalizations of a hypothesis for a positive example
```

```
def minimal_generalizations(h, x):
```

```
    result = list(h)
```

```
    for i in range(n):
```

```
        if result[i] == 'Ø':
```

```
            result[i] = x[i]
```

```
        elif result[i] != x[i] and result[i] != '?':
```

```
            result[i] = '?'
```

```
    return [tuple(result)]
```

```
# Minimal specializations of a hypothesis for a negative example
```

```
def minimal_specializations(h, x, S):
```

```
    results = []
```

```
    for i in range(n):
```

```
        if h[i] == '?':
```

```
            for value in domains[i]:
```

```
                if value != x[i]:
```

```
                    new_h = list(h)
```

```
                    new_h[i] = value
```

```

        new_h = tuple(new_h)

        # Must be more general than at least one S
        if any(more_general(new_h, s) for s in S):
            results.append(new_h)

    return results

# Remove duplicate hypotheses
def unique(hypotheses):
    return list(dict.fromkeys(hypotheses))

# Initial boundaries
S = {tuple(['Ø'] * n)}
G = {tuple(['?'] * n)}

print("\nInitial Specific Boundary S:")
print(S)

print("\nInitial General Boundary G:")
print(G)

# Candidate-Elimination
for index, (x, label) in enumerate(zip(concepts, labels)):

    x = tuple(x)

    if label in ['yes', 'positive', '1', 'true']:

        # Remove inconsistent hypotheses from G

```

$G = \{g \text{ for } g \text{ in } G \text{ if covers}(g, x)\}$

`new_S = set()`

for `s` in `S`:

 if `covers(s, x)`:

`new_S.add(s)`

 else:

 for `h` in `minimal_generalizations(s, x)`:

 if `any(more_general(g, h) for g in G)`:

`new_S.add(h)`

`S = new_S`

Remove `S` hypotheses that are more general than another `S`

`S = {`

`s` for `s` in `S`

 if not `any(`

`s != s2 and more_general(s, s2)`

 for `s2` in `S`

)

`}`

else:

Remove inconsistent hypotheses from `S`

`S = {s for s in S if not covers(s, x)}`

`new_G = set()`

```

for g in G:
    if not covers(g, x):
        new_G.add(g)
    else:
        specializations = minimal_specializations(g, x, S)
        new_G.update(specializations)

```

```

G = unique(list(new_G))

```

```

G = set(G)

```

```

# Remove G hypotheses that are less general than another G

```

```

G = {
    g for g in G
    if not any(
        g != g2 and more_general(g2, g)
        for g2 in G
    )
}

```

```

print("\nAfter Example", index + 1, "(" + label + ")")

```

```

print("S =", list(S))

```

```

print("G =", list(G))

```

```

# Final boundaries

```

```

print("\n=====")

```

```

print("FINAL CANDIDATE-ELIMINATION RESULT")

```

```

print("=====")

```

```

print("\nSpecific Boundary (S):")

```

```

for s in S:

```

```

print(s)

print("\nGeneral Boundary (G):")
for g in G:
    print(g)

# Generate all possible hypotheses from the attribute values
hypothesis_values = [values + ['?'] for values in domains]

all_hypotheses = list(product(*hypothesis_values))

# Find all hypotheses consistent with the complete training data
consistent = []

for h in all_hypotheses:
    valid = True

    for x, label in zip(concepts, labels):

        prediction = covers(h, tuple(x))
        actual = label in ['yes', 'positive', '1', 'true']

        if prediction != actual:
            valid = False
            break

    if valid:
        consistent.append(h)

print("\nAll Consistent Hypotheses (Version Space):")

```

```

for h in consistent:
    print(h)

print("\nTotal Consistent Hypotheses =", len(consistent))

```

Output

Training Dataset:

	day	outlook	temp	humidity	wind	play
D1	Sunny	Hot	High	Weak	No	
D2	Sunny	Hot	High	Strong	No	
D3	Overcast	Hot	High	Weak	Yes	
D4	Rain	Mild	High	Weak	Yes	
D5	Rain	Cool	Normal	Weak	Yes	
D6	Rain	Cool	Normal	Strong	No	
D7	Overcast	Cool	Normal	Strong	Yes	
D8	Sunny	Mild	High	Weak	No	
D9	Sunny	Cool	Normal	Weak	Yes	
D10	Rain	Mild	Normal	Weak	Yes	
D11	Sunny	Mild	Normal	Strong	Yes	
D12	Overcast	Mild	High	Strong	Yes	
D13	Overcast	Hot	Normal	Weak	Yes	
D14	Rain	Mild	High	Strong	No	

Initial Specific Boundary S:

$\{(\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)\}$

Initial General Boundary G:

$\{('?', '?', '?', '?', '?')\}$

After Example 1 (no)

$S = []$

$G = []$

After Example 2 (no)

$S = []$

$G = []$

After Example 3 (yes)

$S = []$

$G = []$

After Example 4 (yes)

$S = []$

$G = []$

After Example 5 (yes)

$S = []$

$G = []$

After Example 6 (no)

$S = []$

$G = []$

After Example 7 (yes)

$S = []$

$G = []$

After Example 8 (no)

S = []

G = []

After Example 9 (yes)

S = []

G = []

After Example 10 (yes)

S = []

G = []

After Example 11 (yes)

S = []

G = []

After Example 12 (yes)

S = []

G = []

After Example 13 (yes)

S = []

G = []

After Example 14 (no)

S = []

G = []

=====

FINAL CANDIDATE-ELIMINATION RESULT

=====

Specific Boundary (S):

General Boundary (G):

All Consistent Hypotheses (Version Space):

Total Consistent Hypotheses = 0

Result

Thus, the Candidate-Elimination algorithm was implemented successfully and the set of hypotheses consistent with the given training examples was obtained